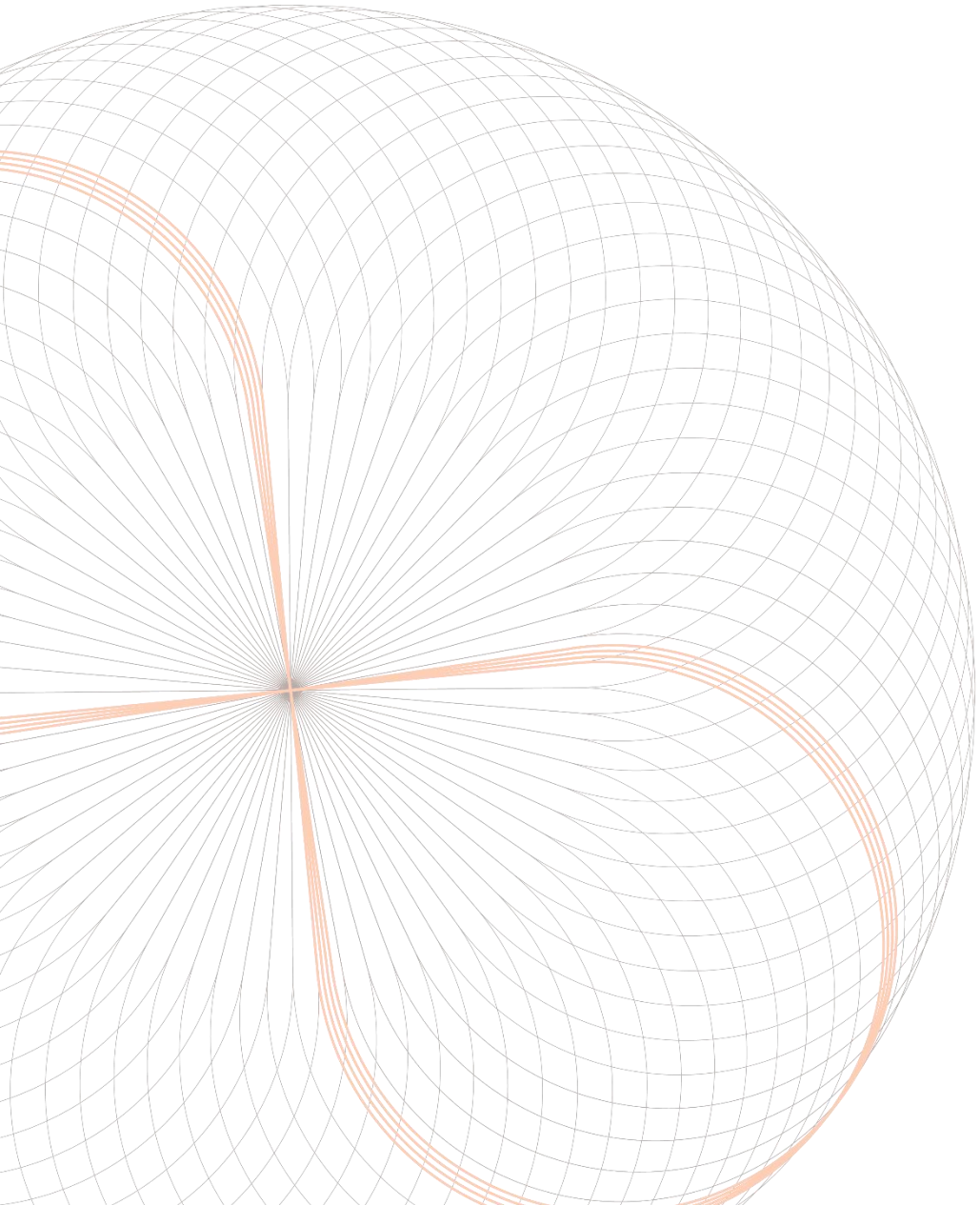
Margin and collateral
intelligence for derivatives.

cassini

Cassini Technical Assessment

QA Automation Engineer

2025



Margin and collateral
intelligence for derivatives.

cassini

Assessment Details

This document provides test scenarios for implementing an API automation testing framework using Python. The scenarios cover various testing aspects, including functional, non-functional, and edge case testing.

Setup Requirements

Base URL Configuration

- JSONPlaceholder: https://JSONplaceholder.typicode.com (No auth required)
- ReqRes: https://reqres.in/api (Simulated responses)
- HTTPBin: https://httpbin.org (HTTP testing service)
- GoRest: https://gorest.co.in/public/v2 (Requires API token)

Authentication Methods

- GoRest API Token: Authorization: Bearer <your_token>
    - Get free token at: https://gorest.co.in/
- HTTPBin Basic Auth: Authorization: Basic <base64_credentials>
- ReqRes: No authentication required (simulated responses)

Core Test Scenarios

1. Authentication & Authorisation Tests

TC_AUTH_001: Valid Authentication (ReqRes)

- Endpoint: POST https://reqres.in/api/login
- Request Body:

JSON

```
{
  "email": "eve.holt@reqres.in",
  "password": "cityslicka"
}
```

- Expected Response: 200 OK with token
- Validation Points:
    - Response contains token field
    - Token is non-empty string
    - Response time < 2 seconds
    - Status code is exactly 200

TC_AUTH_002: Invalid Credentials (ReqRes)

- Endpoint: POST https://reqres.in/api/login
- Request Body:

JSON

```
{
  "email": "eve.holt@reqres.in",
  "password": "wrongpassword"
}
```

- Expected Response: 400 Bad Request
- Validation Points:
    - Error field present in response
    - No token returned
    - Consistent JSON error structure

TC_AUTH_003: Missing Password (ReqRes)

- Endpoint: POST https://reqres.in/api/login
- Request Body:

JSON

```
{
  "email": "eve.holt@reqres.in"
}
```

- Expected Response: 400 Bad Request
- Validation Points:
  - Error message: "Missing password"
  - Response follows standard error format

2. CRUD Operations Tests

TC_CRUD_001: Create Resource – JSONPlaceholder

- Endpoint: POST https://JSONplaceholder.typicode.com/posts
- Request Body:

JSON

```
{
  "title": "Test Post Title",
  "body": "This is a test post body content for automation testing",
  "userId": 1
}
```

- Expected Response: 201 Created
- Validation Points:

- Response contains created resource with ID (101)
- All input fields present in response
- ID field auto-generated
- Response time < 1 second

TC_CRUD_002: Create User – GoRest (Requires Token)

- Endpoint: POST https://gorest.co.in/public/v2/users
- Headers: Authorization: Bearer <your_gorest_token>
- Request Body:

JSON

```
{
  "name": "John Doe Automation",
  "email": "john.doe.automation@example.com",
  "gender": "male",
  "status": "active"
}
```

- Expected Response: 201 Created
- Validation Points:
  - Response contains created user with ID
  - All input fields properly saved
  - Email uniqueness enforced

TC_CRUD_003: Read Single Resource – JSONPlaceholder

- Endpoint: GET https://JSONplaceholder.typicode.com/posts/1
- Expected Response: 200 OK
- Validation Points:

- Complete resource data returned
- Fields: id, title, body, userId present
- Data types: id and userId are integers
- title and body are strings

3. Data Validation Tests

TC_VALID_001: Missing Required Fields – GoRest

- Endpoint: POST https://gorest.co.in/public/v2/users
- Headers: Authorization: Bearer <your_gorest_token>
- Request Body:

JSON

```
{
  "name": "John Doe"
  // Missing required email, gender, status fields
}
```

- Expected Response: 422 Unprocessable Entity
- Validation Points:
  - Error array with specific field validation messages
  - Each missing field identified separately
  - Proper error code (422) for validation failure

TC_VALID_002: Invalid Email Format – GoRest

- Endpoint: POST https://gorest.co.in/public/v2/users
- Headers: Authorization: Bearer <your_gorest_token>

- Request Body:

JSON

```
{
  "name": "John Doe",
  "email": "invalid-email-format",
  "gender": "male",
  "status": "active"
}
```

- Expected Response: 422 Unprocessable Entity
- Validation Points:
  - Email format validation error in response
  - Error message indicates invalid email format
  - Other fields validation passed

TC_VALID_003: Invalid Enum Values - GoRest

- Endpoint: POST https://gorest.co.in/public/v2/users
- Headers: Authorization: Bearer <your_gorest_token>
- Request Body:

JSON

```
{
  "name": "John Doe",
  "email": "john@example.com",
  "gender": "unknown",
  "status": "maybe"
}
```

- Expected Response: 422 Unprocessable Entity

- Validation Points:
    - Invalid enum values rejected
    - Error messages for gender and status fields
    - Valid values suggested in error response

4. Error Handling Tests

TC_ERROR_001: Resource Not Found - JSONPlaceholder

- Endpoint: GET https://JSONplaceholder.typicode.com/posts/99999
- Expected Response: 404 Not Found
- Validation Points:
    - Empty response body: {}
    - HTTP status code 404
    - No sensitive information exposed

TC_ERROR_002: Method Not Allowed - HTTPBin

- Endpoint: DELETE https://httpbin.org/get
- Expected Response: 405 Method Not Allowed
- Validation Points:
    - Method not allowed error
    - Allowed methods may be listed in response
    - Consistent error format

TC_ERROR_003: Invalid URL Path - JSONPlaceholder

- Endpoint: GET https://JSONplaceholder.typicode.com/invalid-endpoint

- Expected Response: 404 Not Found
- Validation Points:
  - Empty response body: {}
  - 404 status code for invalid endpoints
  - Consistent error handling

5. Performance Tests

TC_PERF_001: Response Time Validation - JSONPlaceholder

- Endpoints:
  - GET https://JSONplaceholder.typicode.com/posts (All posts)
  - GET https://JSONplaceholder.typicode.com/posts/1 (Single post)
  - POST https://JSONplaceholder.typicode.com/posts
- Requirements:
  - GET single: < 500ms
  - GET collection: < 1 second
  - POST requests: < 1 second
- Validation Points:
  - Response time measurement and assertion
  - Performance regression detection
  - Network latency consideration

TC_PERF_002: Concurrent Request Handling - HTTPBin

- Endpoint: GET https://httpbin.org/delay/1
- Load: 10 concurrent requests
- Expected Behavior: All requests complete in ~1 second

- Validation Points:
    - No failed requests due to concurrency
    - Response times remain consistent
    - Server handles concurrent load

TC_PERF_003: Pagination Performance - JSONPlaceholder

- Endpoints:
    - GET https://JSONplaceholder.typicode.com/posts?_page=1&_limit=20
    - GET https://JSONplaceholder.typicode.com/posts?_page=2&_limit=20
    - GET https://JSONplaceholder.typicode.com/posts?_page=5&_limit=20
- Expected Response: 200 OK with paginated results
- Validation Points:
    - Consistent response times across pages
    - Proper pagination headers (if available)
    - No performance degradation on later pages

TC_PERF_004: Stress Test - HTTPBin Echo

- Endpoint: POST https://httpbin.org/post
- Load: 50 requests in 30 seconds
- Request Body: Medium-sized JSON payload (1KB)
- Validation Points:
    - Success rate > 95%
    - Average response time remains stable

- No error rate increase under load

8. Edge Cases & Boundary Tests

TC_EDGE_001: Empty Request Body

- Endpoint: POST /users
- Request Body: {}
- Expected Response: 400 Bad Request
- Validation Points:
  - Proper validation error messages
  - No server crashes

TC_EDGE_002: Maximum String Lengths

- Endpoint: POST /users
- Request Body: Fields at maximum allowed length
- Expected Response: 201 Created
- Validation Points:
  - Data saved correctly
  - No truncation without warning

TC_EDGE_003: Unicode and Special Characters

- Endpoint: POST /users
- Request Body:

JSON

```
{
```

"name": "José María González-Pérez £$%^",
"email": "josé@müller.com"
}

- Expected Response: 201 Created
- Validation Points:
  - Unicode characters properly handled

Framework Implementation Requirements

Framework Structure

- Industry standard

Key Features to Implement

1. Configurable Environment Management
2. Automatic Request/Response Logging
3. Schema Validation for Responses
4. Test Data Management and Cleanup
5. Retry Mechanisms for Flaky Tests
6. Parallel Test Execution Support
7. Comprehensive Reporting with Logs
8. CI/CD Pipeline Integration

Execution Strategy

- Smoke Tests: Critical path scenarios
- Regression Suite: All functional tests
- Performance Tests: Separate execution with monitoring

Expected Deliverables

1. Complete Test Framework
2. All scenarios are implemented as automated tests. (if not at least 2 in each section)
3. Documentation: Setup guide, execution instructions, and maintenance guide (Optional)
4. CI/CD Integration: Pipeline configuration for automated execution (Optional)
5. Reporting: Test results
6. Data Management: Test data setup and cleanup procedures

Success Metrics

- Test Coverage: 100% of API endpoints covered (or as mentioned in the point 2 in the above section)
- Execution Time: Full regression suite under 10 minutes
- Reliability: 95%+ test stability (non-flaky)
- Maintenance: Easy addition of new test scenarios
- Reporting: Clear and actionable test results